Kleisli Composition

Kleisli · Compose parseInt → enrichUser → authorize · Build with Waqar

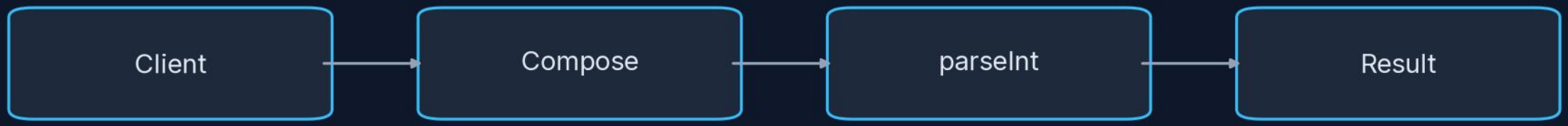
Today's Hook

- Today's force: Compose parseInt → enrichUser → authorize
- We earn Kleisli by feeling the naive pain first.
- Demo main prints a real JDK 25 trace (happy + failure).
- Key types: Kleisli, AuthPipe

The Problem

- Scenario: Compose parseInt → enrichUser → authorize
- Constraints: solve «Compose parseInt → enrichUser → authorize» with Kleisli — keep the demo runnable and the failure path observable.
- Naive pain: Naive version fails to cleanly solve: Compose parseInt → enrichUser → authorize
- Success looks like: allow:7

Structure



Kleisli: Compose parseInt → enrichUser → authorize

Real-World Scenario

- Running example: Compose parseInt → enrichUser → authorize
- Key types: Kleisli, AuthPipe
- Happy-path stdout cue: allow:7
- Failure-path stdout cue: Optional.empty

Naive Code

```
package com.buildwithwaqar.patterns.functional.kleisli.problem;

import java.util.*;

public final class NaiveAuth {
  public static String auth(String raw){ int id=Integer.parseInt(raw); return id>0? "allow:"+id: "deny"; }
}
```

Why Naive Hurts

- Naive version fails to cleanly solve: Compose parseInt → enrichUser → authorize
- Without Kleisli, call sites absorb the wrong details.
- Failure mode to show on camera: Optional.empty
- New requirements force shotgun edits across the naive code.

Solution

```
package com.buildwithwaqar.patterns.functional.kleisli.problem;

import java.util.*;
import java.util.function.*;

final class Kleisli<A,B> {
  private final Function<A, Optional<B>> run;
  Kleisli(Function<A, Optional<B>> run){ this.run=run; }
  static <A,B> Kleisli<A,B> of(Function<A, Optional<B>> f){ return new Kleisli<>(f); }
  <C> Kleisli<A,C> andThen(Kleisli<B,C> next){ return new Kleisli<>(a -> run.apply(a).flatMap(next.run)); }
  Optional<B> apply(A a){ return run.apply(a); }
}
final class AuthPipe {
  static final Kleisli<String,Integer> parse = Kleisli.of(s -> { try { return Optional.of(Integer.parseInt(s)); } catch(Exception e){ return Optional.empty(); } });
  static final Kleisli<Integer,Integer> enrich = Kleisli.of(id -> id>0? Optional.of(id+1000): Optional.empty());
  static final Kleisli<Integer,String> authorize = Kleisli.of(id -> Optional.of("allow:"+id));
  static final Kleisli<String,String> pipeline = parse.andThen(enrich).andThen(authorize);
}
```

Runnable main

```
package com.buildwithwaqar.patterns.functional.kleisli.problem;
import java.util.*; import java.util.function.*;
public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Kleisli: parse → enrich → authorize ===");
    System.out.println("--- naive ---"); System.out.println(NaiveAuth.auth("7"));
System.out.println("--- kleisli ---");
System.out.println(AuthPipe.pipeline.apply("7"));
System.out.println(AuthPipe.pipeline.apply("x"));
System.out.println(AuthPipe.pipeline.apply("-1"));
  }
}
```

Walkthrough Output

```
=== Kleisli: parse → enrich → authorize ===
--- naive ---
allow:7
--- kleisli ---
Optional[allow:1007]
Optional.empty
Optional.empty
```

Step-by-Step Trace

- Match stdout to stages — look for: allow:7
- Then the injected failure: Optional.empty
- Re-run: java -cp ... 57-kleisli ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```
package com.buildwithwaqar.patterns.functional.kleisli.problem;

import java.util.*;
import java.util.function.*;

final class Kleisli<A,B> {
  private final Function<A, Optional<B>> run;
  Kleisli(Function<A, Optional<B>> run){ this.run=run; }
  static <A,B> Kleisli<A,B> of(Function<A, Optional<B>> f){ return new Kleisli<>(f); }
  <C> Kleisli<A,C> andThen(Kleisli<B,C> next){ return new Kleisli<>(a -> run.apply(a).flatMap(next.run)); }
  Optional<B> apply(A a){ return run.apply(a); }
}
final class AuthPipe {
  static final Kleisli<String,Integer> parse = Kleisli.of(s -> { try { return Optional.of(Integer.parseInt(s)); } catch(Exception e){ return Optional.empty(); } });
  static final Kleisli<Integer,Integer> enrich = Kleisli.of(id -> id>0? Optional.of(id+1000): Optional.empty());
  static final Kleisli<Integer,String> authorize = Kleisli.of(id -> Optional.of("allow:"+id));
  static final Kleisli<String,String> pipeline = parse.andThen(enrich).andThen(authorize);
}
```

Trade-offs

- Use Kleisli when «Compose parseInt → enrichUser → authorize» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Kleisli in product language.

Related Patterns

- Name the pattern usually confused with Kleisli and separate the forces.
- Sketch the roles for «Compose parseInt → enrichUser → authorize» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Compose parseInt → enrichUser → authorize
- Naive pain → Kleisli boundary → Demo stdout.
- Remember the failure cue: Optional.empty
- Break a ProblemTest assertion, watch it fail, restore it.